

Insiemi Disgiunti

$$\mathcal{S} = \{S_1, \dots, S_m\}$$

S_i insieme di oggetti

$$S_i = \{x_{i1}, x_{i2}, \dots, x_{i|S_i|}\}$$

$$S_i \cap S_j = \emptyset \quad \text{e} \quad i \neq j$$

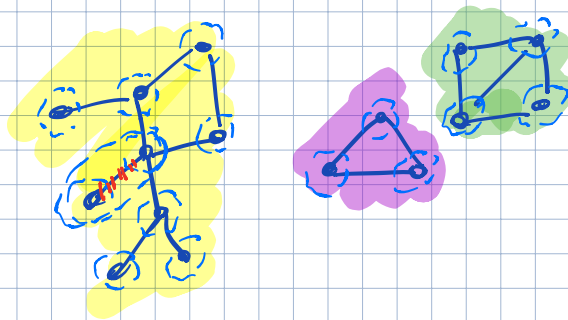
$$S_i \neq \emptyset$$

- Make(x) costruire $\{x\}$
- Find(x) restituire S_i t.c. $x \in S_i$
- Union(x, y) se $x \in S_i$ e $y \in S_j$ con $i \neq j$
sostituire S_i e S_j con
 $S_i \cup S_j$

Esempio

$$G = (V, E)$$

Componenti
connesse



$\Rightarrow$ Costruisco $\{x\}$ per ogni $x \in V$

$\Rightarrow$ Per ogni $\{x, y\} \in E$ Union (x, y)

Alla fine ho un insieme di nodi per ogni
componente connessa

$\Theta(|V|)$ MAKE

$O(|E|)$ UNION e FIND



Il costo effettivo

dipende da come sono
implementati gli insiemi
disgiunti.



Nota

Analizzeremo il costo di m operazioni
complessive (Make, Union, Find) di cui m
operazioni MAKE

$$m = \underbrace{m}_{\text{Make}} + \underbrace{u}_{\text{Union}} + \underbrace{f}_{\text{Find}}$$

$$m \geq m$$

$$u \leq m-1$$

Implementazioni

Liste conc.

Alberi

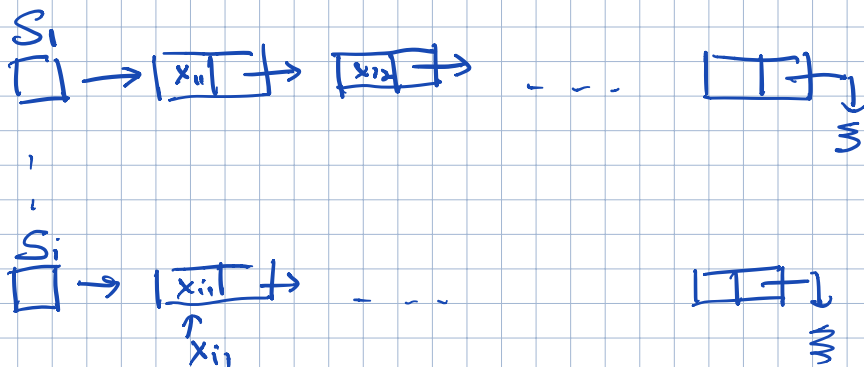
Ottimizzazioni

- Weighted Union

- Union by rank
Path Compression

Applicazione: Minimum Spanning Tree

MUF con Liste



x è un oggetto record di una lista concatenata

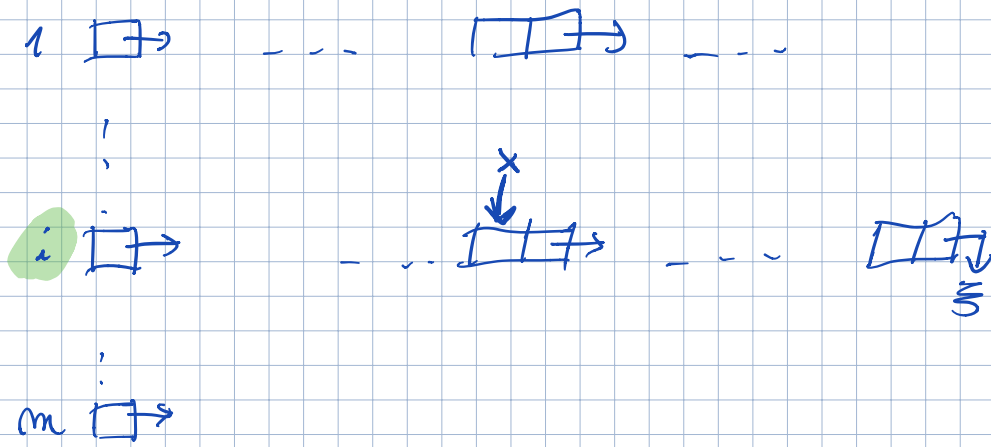
x . key

chiave ... info ..

x . next

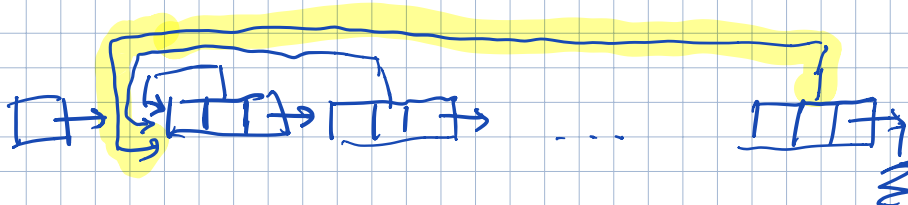
punt. al successivo

Find (x) ?



Troppo costoso senza altre inform.

- In ogni elemento metterlo anche un puntatore al rappresentante della lista (il primo elemento della lista)



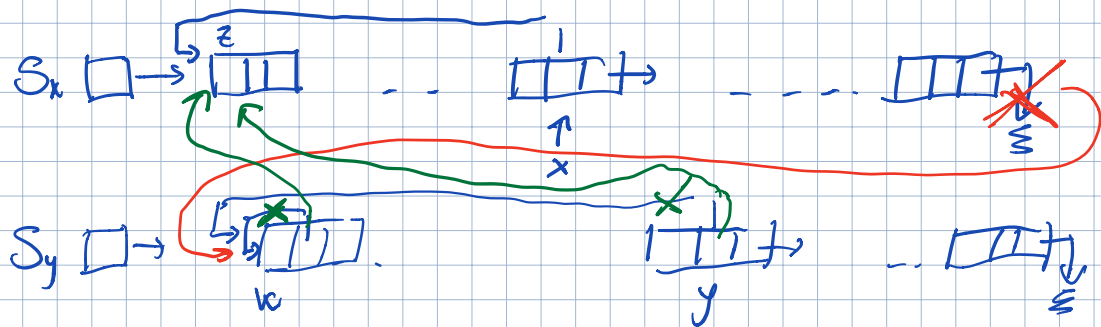
x.key

x.rep

x.next

puntatore al rappresent.

Union (x, y) ?



Concateniamo le liste di y e quelle di x

- 1 • scandisco tutta S_x per arrivare all'ultimo —
- 2 • scandisco tutta S_y per modificare tutti i rappresentanti —

$$O(|S_x| + |S_y|)$$

Posso risparmiare una delle due scandiscie?

Servo un puntatore all'ultimo elem.

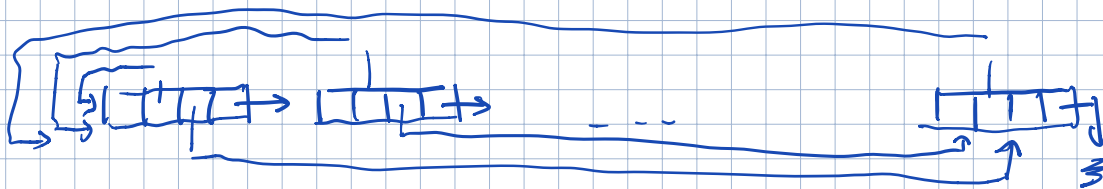
Σ
campo aggiuntivo
negli oggetti delle liste

$x.key$

$x.rep$

$x.next$

$x.last$ ← Viene aggiornato solo nel reppres.



Per eseguire $\text{Union}(x, y)$ scendiamo solo S_y

$\text{Make}(x)$ {
 % x è già del tipo corretto
 % $x.\text{key}$ è già inizializzato

$x.\text{top} \leftarrow x$
 $x.\text{next} \leftarrow \text{NIL}$
 $x.\text{last} \leftarrow x$
 return x



}

$\text{Find}(x)$ {
 return $x.\text{top}$
 }

$\text{Union}(x, y)$ {
 $z \leftarrow \text{Find}(x)$
 $w \leftarrow \text{Find}(y)$
 if $(z \neq w)$ {
 $\text{Link}(z, w)$
 }

Link (z, w) {

z, w sono rappres.
di 2 liste $z \neq w$

z.list.next $\leftarrow$ w

z.list $\leftarrow$ w.list

while (w $\neq$ nil) {

w.tail $\leftarrow$ z

w $\leftarrow$ w.next

}

}

Costi

1 Make $\mathcal{O}(1)$

1 Find $\mathcal{O}(1)$

1 Union $\mathcal{O}(|L_y|)$ lung. delle 2^e liste

m operazioni MUF di cui n Make

Make $\mathcal{O}(1) \cdot n \rightarrow \mathcal{O}(n)$

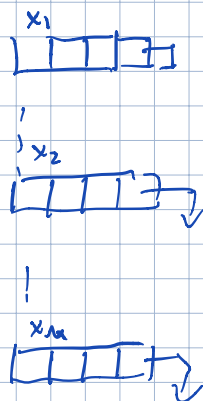
Find $\mathcal{O}(1) \cdot \underbrace{f_m}_m \rightarrow \mathcal{O}(1) \cdot \mathcal{O}(m) = \mathcal{O}(m)$

Union $\underbrace{\mathcal{O}(m)}_{\text{lung. 2^e lista}} \cdot \underbrace{u}_{\leq m-1} \rightarrow \mathcal{O}(m) \cdot \mathcal{O}(m) = \mathcal{O}(m^2)$

$\mathcal{O}(m + m^2)$

$m = n + u + f$

È possibile arrivare al caso peggiore $\Theta(n + n^2)$?
 sì.



Union (x_2, x_1)

Union (x_3, x_1)

Union (x_4, x_1)

!

Union (x_i, x_1) $\leftarrow$ 2^a lista range $i-1$
 $\Theta(i)$

!

Union (x_n, x_1)

Costo complessivo

$$\sum_{i=2}^n \Theta(i) = \frac{n(n+1)}{2} - 1 = \Theta(n^2)$$

Weighted Union

Per evitare il caso peggiore aggiungo

x . length aggiornato solo nel rep.

Nelle union metto come seconda
 lista la più corta

Make(x) {

 x.top ← x

 x.next ← nil

 x.last ← x

 x.length ← 1

 return x

}

Find(x) {

 return x.top

}

```

WeigUnion(x, y) {
    z ← Find(x)
    w ← Find(y)
    if (z ≠ w) {
        if (z.length ≥ w.length) {
            WLink(z, w)
        } else {
            WLink(w, z)
        }
    }
}

```

```

WLink(z, w) {
    z.Next.next ← w
    z.Next ← w.Next
    z.length ← z.length + w.length
    while (w ≠ nil) {
        w.Next ← z
        w ← w.Next
    }
}

```

Cost:

1	Make	$\Theta(1)$	
1	Find	$\Theta(1)$	
1	Union	$\Theta(L_2)$	$ L_2 = \min(L_x , L_y)$

m operazioni HUF di cui n Hake usando
Weighted Union

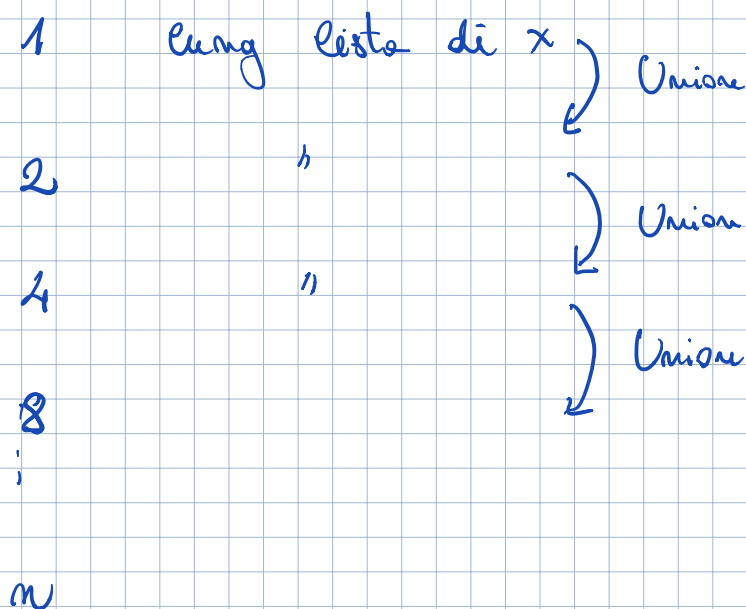
Posso valutare il costo di tutte le Union
analizzando quanti punt. $x.rep$ vengo
modificati complessivamente

Concentriamoci su un oggetto x

Quante volte al più modifico $x.rep$?
 $\sum$

Quante volte al più x può finire nella
seconda lista?
 $\sum$

Quante volte al più riesco ad unire
la lista in cui sta x ad una lista più
lunga (o stessa lung)?



3

Riesco a modificare x. rep
al più $O(\log n)$ volte

3

Quindi in tutti i possibili x
 $O(n \log n)$

Costo complessivo Weighted Union

$$O(n \log n)$$

Compless. m operazioni MUF di cui n Merge
con Weighted Union costano

$$O(m + n \log n)$$

3
find